

Delegato: Locality-Aware Atomic Memory Operations on Chiplets

Víctor Soria-Pardos
victor.soria@bsc.es
Barcelona Supercomputing Center

Adrià Armejach
adria.armejach@upc.edu
Univseritat Politècnica de Catalunya,
Barcelona Supercomputing Center

Tiago Mück
tiago.muck@arm.com
Arm

Darío Suárez Gracia
dario@unizar.es
Universidad de Zaragoza

José A. Joao
jose.joao@arm.com
Arm

Miquel Moretó
miquel.moreto@upc.edu
Univseritat Politècnica de Catalunya,
Barcelona Supercomputing Center

Abstract

The irruption of chiplet-based architectures has been a game changer, enabling higher transistor integration and core counts in a single socket. However, chiplets impose higher and non-uniform memory access (NUMA) latencies than monolithic integration. This harms the efficiency of atomic memory operations (AMOs), which are fundamental to implementing fine-grained synchronization and concurrent data structures on large systems. AMOs are executed either near the core (near) or at a remote location within the cache hierarchy (far). On near AMOs, the core's private cache fetches the target cache line in exclusiveness to modify it locally. Near AMOs cause significant data movement between private caches, especially harming parallel applications' performance on chiplet-based architectures. Alternatively, far AMOs can alleviate the communication overhead by reducing data movement between processing elements. However, current multicore architectures only support one type of far AMO, which sends all updates to a single serialization point (centralized AMOs).

This work introduces two new types of far AMOs, delegated and migrating, that execute AMOs remotely without centralizing updates in a single point of the cache hierarchy. Combining centralized, delegated, and migrating AMOs allows the directory to select the best location to execute AMOs. Moreover, we propose Delegato, a tracing optimization to effectively transport usage information from private caches to the directory to predict the best atomic type to issue accurately. Additionally, we design a simple predictor on top of Delegato that seamlessly selects the best placement to perform AMOs based on the data access pattern and usage activity of cores. Our evaluation using gem5 shows that Delegato can speed up applications on average by 1.07× over centralized AMOs and by 1.13× over the state-of-the-art AMO predictor.

1 Introduction

Chiplet-based architectures have surpassed monolithic systems and system-on-chip (SoC) designs in the semiconductor market [68, 74]. Chip vendors have adopted chiplet-based designs as a cost-effective alternative for scaling multicores without increasing die size, enhancing yield and reducing costs [68, 116]. Compared to monolithic systems, the main drawback is the introduction of interposers, which increase the communication latency between chiplets and

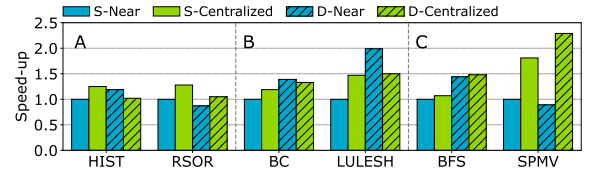


Figure 1: Speed-up of single-chiplet (S) and dual-chiplet (D) Grace CPU configurations using near and centralized AMOs. Normalized w.r.t. single-chiplet near AMOs execution time. NUMA effects [33, 87]. This harms the performance of latency-sensitive operations such as Atomic Memory Operations (AMOs), which are extensively used to synchronize parallel workloads.

All modern instruction set architectures (ISAs) feature AMOs to perform atomic read-modify-write (RMW) operations on memory, such as Armv9 [49], x86 [53], RISC-V [111], and Nvidia PTX [94]. AMOs are fundamental for updating shared data directly or creating high-level synchronization mechanisms such as mutually exclusive locks, barriers, and lock-free data structures. ISAs continuously extend their AMO support to include more complex primitives, reducing the programmers' burden. For instance, x86 has recently introduced conditional increment AMOs (CMPCCXADD) [53], and Arm has introduced floating-point AMO extensions (LSFE) [51].

AMOs require dedicated support from coherence protocols to ensure correct cache behavior. Most systems perform AMOs within the private levels of the cache hierarchy by acquiring exclusive permission of the targeted cache line, performing the RMW operation locally (*near* AMOs) [4]. Alternatively, *centralized* AMOs send all updates for a cache line to a single designated location, typically the corresponding directory slice, where the RMW operation is performed remotely [48, 70, 99]. Therefore, centralized AMOs aggregate updates at the directory, avoiding cache line migration across multiple private caches [29, 102]. However, centralized AMOs can trigger invalidations that may cause future data misses.

Figure 1 illustrates the performance impact of near and centralized AMOs on six applications running on NVIDIA's Grace CPU Superchip [93], evaluated under single-chiplet and dual-chiplet configurations. The two chiplets are interconnected via NVLink, which offers high bandwidth but incurs significant latency. GCS supports cross-chiplet atomics, which send AMOs remotely to the directory where the cache line resides, reducing data movement over NVLink. In the single-chiplet configuration, centralized AMOs consistently outperform near AMOs across all benchmarks. However, in dual-chiplet configurations, we observe three distinct behaviors, labeled A, B, and C in Figure 1. In category A, centralized AMOs exhibit lower performance in a dual-chip setup than in a single-chiplet

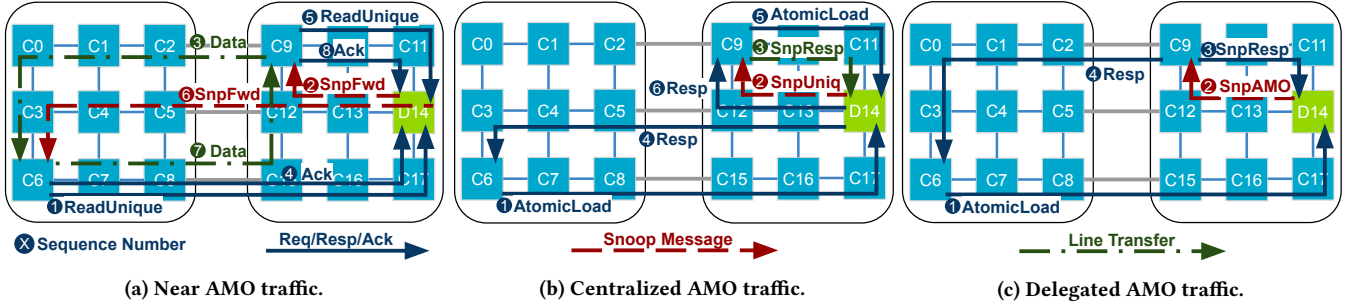


Figure 2: Simplified example comparing AMOs under three schemes on a chiplet-based system. Core 6 (C6) performs an AMO on a cache line owned by Core 9 (C9), tracked by directory 14 (D14). Then, C9 performs another AMO to the same line. (a) With near AMOs, C6 and C9 ping-pong the line between chiplets. (b) With centralized AMOs, D14 invalidates C9, and then performs AMOs from C6 and C9. (c) With delegated AMOs, all updates are performed in C6 without requiring invalidations.

setup. Category B shows applications for which centralized AMOs improve performance compared to a single-chiplet configuration, but near AMOs perform better. Finally, category C represents cases where centralized AMOs in the dual-chiplet configuration yield the best overall performance. Moving from single to dual-chiplet reduces the suitability of centralized AMOs in four out of six applications, motivating the need for more efficient AMO execution strategies in chiplet-based systems.

Although centralized AMOs usually reduce the cost of updates, they may introduce coherence misses [30, 92] when invalidating the cache-line owner. Moreover, centralized AMOs cause significant interconnect traffic because every update must be sent to a shared, fixed per-line location. Figure 2 illustrates this scenario by showing the interconnect hops required when a core from a different chiplet performs a near AMO (Figure 2a) and a centralized AMO (Figure 2b) on a line currently owned by another core.

This paper introduces two new types of far AMOs, *delegated* and *migrating* AMOs. Delegated AMOs forward the RMW operation to the cache line owner, who executes it locally without requiring an invalidation or a cache line transfer. Delegated AMOs effectively reduce coherence misses by removing superfluous invalidations, as shown in Figure 2c. Alternatively, migrating AMOs transfer the cache line ownership to the AMO requester. Integrating centralized, delegated, and migrating AMOs enables placement decisions at the directory level through policies, programmer hints, or predictors; potentially reducing interconnect traffic and overall overhead when performing AMOs.

Moreover, we find that predicting the best type of far AMO at the directory level is difficult due to the lack of information available. Thus, we propose *Delegato*, a tracing mechanism that effectively transports reuse information from private caches to the directory. We demonstrate Delegato’s utility by designing an AMO predictor that selects the best placement for AMO based on Delegato feedback information. This Delegato-based predictor improves AMOs’ performance across multiple well-known benchmarks.

The main contributions of this work are:

- We propose two new types of far AMOs, delegated and migrating, to effectively manage data movement across chiplets. Both transactions enable far AMOs without requiring an arithmetic-logic unit (ALU) at the directory.
- We investigate the design space of static policies for far AMOs at the directory when selecting between centralized, delegated, and migrating AMOs.

- We propose Delegato, a trace mechanism to enable accurate predictions at the directory. We design a predictor on top of Delegato to select the location for executing AMOs.
- We evaluate all proposals using gem5-23 [75]. The Delegato-based predictor achieves a performance improvement of 1.13× over the state-of-the-art AMO predictor and 1.07× over centralized AMOs.

2 Background

2.1 AMOs in Modern Systems

AMOs implement atomic RMW primitives (e.g., compare-and-swap, fetch-and-add, etc.). However, AMOs are known for their low throughput and high latency, imposing significant overhead and serialization on parallel applications [56, 57]. Thus, modern systems have incorporated different mechanisms to execute AMOs efficiently:

Near AMOs: L1D caches perform the RMW operations locally. Therefore, the cache coherence protocol must enforce that the L1D cache has exclusive access to the cache line before performing any update. If other private caches have a copy of the targeted line, a structure acting as the point-of-coherence of the system, such as a directory¹, sends invalidation messages to all sharers. When a single private cache owns the line, modern implementations of cache coherence protocols can efficiently resolve the transaction in only three hops because the owner sends the cache line directly to the requester, as shown in Fig. 2a. Future AMOs performed in the same L1D cache will have reduced latency as long as the requester holds the exclusive copy of the cache line. Thus, near AMOs are especially suitable for updating shared data with locality [102]. Nevertheless, near AMOs introduce significant interconnect traffic because cache ownership moves to the L1D of the updater. This data movement pattern is especially harmful when there is a long latency link between the requester and the cache line owner.

Centralized AMOs: The AMOs are sent to the serialization point using an AtomicLoad or AtomicStore request² [49], which performs the RMW at cache line granularity. Typically, the memory controller, or the directory (our baseline), manages this task. When a copy of the cache line is in other private caches, the directory must recover ownership by sending invalidations to the sharers, which can take up to four hops to complete a request. When the line has no sharers and the directory also includes a last-level cache (LLC)

¹This work assumes a many-core system where accesses are interleaved across multiple directory structures to enable scalability.

²AtomicLoads return the previous value stored in memory, AtomicStores do not.

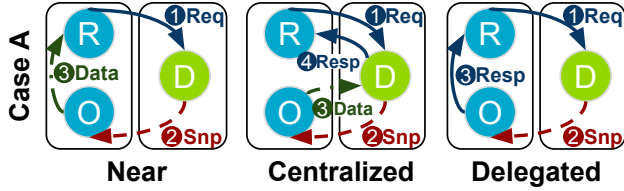


Figure 3: Critical path msgs. between requester (R), directory (D), and owner (O) for near, centralized, and delegated AMOs.

to hold the modified copy of the cache line, subsequent centralized AMOs can be efficiently completed in only two hops, as shown in Fig. 2b. Centralized AMOs have higher latency than near AMOs, but they can achieve higher throughput when multiple updaters exist. On chiplet-based architectures with long latency interconnections, centralized AMOs prevent the expensive data movement of near AMOs that ping-pong lines between chiplets.

Hybrid Schemes: DynAMO [102] is a hybrid scheme that improves overall AMO throughput by selecting between near and centralized AMOs. DynAMO uses a predictor at the L1D, which selects the best placement for atomic RMW operations based on past reuse information. The predictor consists of a table that stores information about recently accessed cache lines by an AMO. For every cache line fetched by a near AMO, the DynAMO predictor tracks if the line is reused. Lines without reuse will be sent to the LLC to perform centralized AMOs. However, DynAMO has some limitations: (i) is very conservative, performing near AMO for most cases and missing optimization opportunities; (ii) uses only local L1D information, lacking information of the whole system; (iii) hybrid schemes cannot reduce coherence misses.

In contrast, we present in Section 4 two alternative far AMOs, delegated and migrating. Figure 2c contrasts the delegated AMO approach proposed in this paper with the near and centralized ones. The AMO is forwarded to the cache line owner, who executes it locally, thus eliminating invalidation and cache line transfer interconnect traffic. The traffic of migrating AMOs is equivalent to that of near AMOs (Figure 2a).

2.2 Alternatives to AMOs

Software Alternatives: There are three popular schemes: *Privatization* and *Software Delegation*. Privatization is the most popular approach, as it exploits commutativity to buffer updates into thread-private storage and then produces the final value. Privatization is commonly used to implement reduction variables efficiently and is supported in most parallel languages [14, 21–23, 28, 34, 58, 88, 117, 117]. However, privatized data increases the memory footprint, introducing cache pollution and consuming more bandwidth. Software delegation [18, 19, 31, 42, 43, 45, 46] divides shared data among threads, so only a single thread would update each memory position. To reduce data movement, software delegation is common in NUMA systems [18, 19] and message-passing architectures [112]. Nevertheless, software delegation introduces extra serialization.

Hardware Alternatives: Other prior work has focused on exploiting the commutative nature of StoreAtomics to delay and reorder the AMOs on the cache hierarchy [7, 24, 36, 119, 119]. However, StoreAtomics represent a relatively small fraction of the total amount of AMOs performed by parallel applications.

3 Centralized AMO Shortcomings

This section analyzes why centralized AMOs fail to provide low-latency and efficient atomic RMW primitives. We cover general problems in every multicore architecture and specific cases on chiplet-based designs and NUMA systems.

3.1 Interconnect Traffic

Despite centralized AMOs avoiding line ping-pong among private caches, they still generate significant interconnect traffic, as all updates are performed at the directory level. Furthermore, most workloads that perform AMOs also have regular loads and stores that conflict with AMOs, so cache lines migrate between private caches despite centralizing AMOs. Alternatively, near AMOs can reduce interconnect traffic as several updates can be performed locally before eviction or invalidation [5]. However, near AMOs cause cache line migration, which consumes significant bandwidth from the network-on-chip (NoC). This traffic is problematic when messages traverse the interconnection between chiplets, as they incur a significant latency penalty and consume a substantial portion of the limited bandwidth.

There are specific pathological scenarios where centralized AMOs perform poorly. Figure 3 compares critical path message flow of near, centralized, and delegated AMOs when both the owner and the requester are in the same chiplet, while the target cache line is managed by a directory in a different chiplet (Case A; the other cases are covered in Section 4.3). All three schemes perform the request/snoop sequence of messages, crossing the interconnection twice. While near and delegated AMOs require one additional interchiplet message to complete the transaction, centralized AMOs require two interchiplet messages to transfer the cache line and respond to the AMO request. Thus, completing a centralized AMO requires four high-latency messages, while the near/delegated cases have only two high-latency messages.

3.2 Coherence Misses

Coherence misses occur in multiprocessor systems when a cache miss happens due to invalidations or downgrades rather than an actual absence of the data in the cache [30, 92]. Coherence misses significantly affect compare-and-swap (CAS) operations. In CAS, the RMW is not always guaranteed to succeed, as the compared data may differ. When CAS is performed in the private cache and multiple cores contend for the same line, they may invalidate the owner while still trying to update the line [56, 57]. Meanwhile, when CAS operations are performed at the directory, they are more likely to fail because the probability of a newer data update increases with the operation latency [57]. Furthermore, some software optimizations, such as co-locating locks and data [62, 79, 102, 118], data locality optimizations [9, 11], and lock-free data structures [47, 106, 109], may further increase the coherence misses. Both near and centralized AMOs generate coherence misses by sending invalidations.

3.3 Area Cost

Centralized AMOs perform RMW operations in the directory, requiring an ALU that introduces area overheads. Most RMW operations are practical, covering simple bit-wise and integer arithmetic.

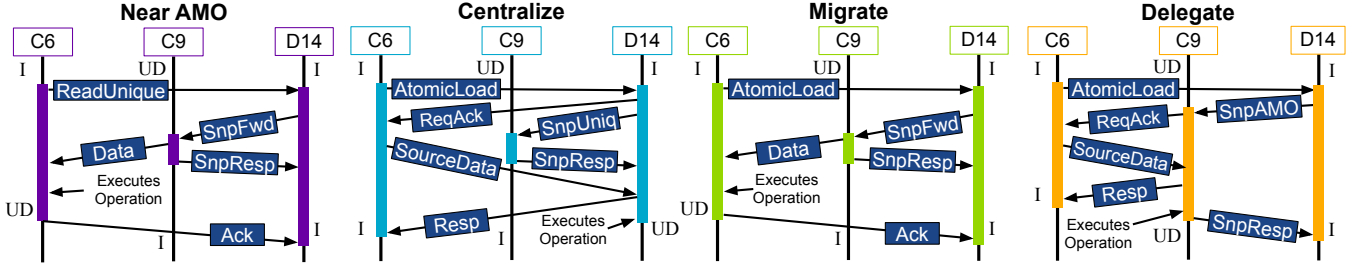


Figure 4: Comparison of four AMO transaction flows: Near, Centralize, Migrate, and Delegate. Core 6 (C6) performs an AMO on a cache line owned by Core 9 (C9), and tracked by Directory 14 (D14).

However, ISAs are introducing more complex operations, such as floating-point addition, requiring more complex ALUs.

We have extended an atomic ALU [35] with support for floating-point min, max, and add operations to quantify its area overhead. We used the 7nm technology node from TSMC, targeting a clock timing of 2GHz. We used the Cadence Genus and Innovus tools v19.11 for the logical synthesis and place-and-route. The ALU occupies $2894 \mu m^2$, from which 85.4% corresponds to the floating-point RMW operations and 14.5% to the bit-wise and simple arithmetic logic. For instance, an Arm Neoverse V1 core tile has an estimated area size of $2.52 mm^2$ in a 7nm process [16]. Thus, adding one floating-point-capable ALU increases a 0.11% the core tile area.

4 Extending AMOs: Delegate and Migrate

This section introduces *Delegate* and *Migrate* transactions. Thus, expanding the directory’s design space with new possible policies to manage how *far* AMOs are executed. Furthermore, we introduce a Chiplet-aware AMO policy, which minimizes chiplet transfer using the locations of the line owner and requester.

4.1 Transaction Specification

Delegate and Migrate are two new coherence transactions complementing Centralized AMO. All three transactions can be triggered upon reception of an AtomicLoad or AtomicStore request. The use of the same request enables flexible remote AMO execution, allowing the directory to select the best location to perform AMOs and reducing data movement. Figure 4 compares Near, Centralize, Delegate, and Migrate message flows.

Near/Centralize: When performing a near AMO, the L1D issues a ReadUnique transaction. Then, the directory invalidates the owner (SnpFwd message), which transfers the line to the requester and acknowledges the directory with a SnpResp. Finally, the requester performs the RMW operation on the L1D upon receiving the data in the Unique state and acknowledges receipt of the data.

On the other hand, Centralized AMOs issue an AtomicLoad, which triggers the directory to invalidate the line owner, denoted by the UD³ state, acquiring exclusive access permission (SnpUnique). In parallel, the directory sends a ReqAck message to the requester, asking for the data operands of the RMW operation. Once the directory receives the line, it sends the original data (Resp) to the requester. The RMW operation is performed using the directory’s ALU only when the data operands and the cache line have arrived.

Migrate: The Migrate transaction has the same transaction flow as a near AMO, transferring the line to the requester. However, an AtomicLoad request is used rather than a ReadUnique, exposing the underlying primitive that triggered the request, which makes near AMOs distinguishable from Stores for the directory. Migrating AMOs can implement near AMOs, but not vice versa. For example, migrating AMOs enables caching the line at intermediate cache levels (i.e., the L2), which may reduce L1D pollution.

Delegate: In the case of a delegate atomic transaction, the directory sends an SnpAMO message to the cache line owner with the RMW operation. Then, the owner answers by sending the ReqAck message to the requester, and in case of an AtomicLoad operation, the Resp message with the original data. The requester sends the data operands to the line owner, which executes the RMW locally. Finally, the line owner notifies the directory of the completion of the snoop transaction. Delegate transactions are not restricted to single-owner scenarios; they can also be applied when multiple sharers exist. In such cases, the directory must invalidate all sharers except one, which would become the new owner responsible for performing the RMW operation. However, such implementations are out of the scope of this paper.

Implementing Migrate and Delegate transactions has multiple implications. First, an ALU must also be placed at the L2, we account for this cost in Section 6.8. Secondly, Migrate and Delegate transactions can be triggered by multiple mechanisms, such as programmers’ hints, specialized ISA instructions, or system configuration. Furthermore, a Delegate transaction can target memory-mapped registers or IO. However, this paper only considers directory AMO policies and AMO predictors as triggers for both transactions.

4.2 Directory Static AMO Policies

Introducing Delegate and Migrate transactions enables the directory to select the best placement for each AMO. Thus, the decision is whether to execute the AMO locally in the directory (Centralized), forward it to the owner (Delegated), or upgrade the requester’s permissions (Migrating). We introduce static AMO policies for the directory, which make placement decisions based only on the cache block coherence state. We assume the directory also contains a LLC slice and can have the cache in one of the following states: Unique Clean (UC) or Unique Dirty (UD), when a cache block is present only at the LLC; Remote Shared Clean (RSC), Remote Shared Dirty (RSD), or Remote Unique (RU), when the cache block is present at an upstream cache in SC, SD, or U state, respectively; and Invalid (I). Based on the LLC/directory state, each static policy selects the type of AMO transaction executed: Migrate (M), Centralize (C), or

³We use the AMBA CHI terminology for cache states: UD (Unique Dirty), SD (Shared Dirty), UC (Unique Clean), SC (Shared Clean), and I (invalid), equivalent to MOESI.

Table 1: Static AMO policies based on LLC/directory cache block state. Migrate (M), Centralize (C), Delegate (D).

	Policy Name	UC/UD	RSC	RSD	RU	I
Existing	All Central	C	C	C	C	C
	All Migrate	M	M	M	M	M
Proposed	Present Central	C	C	C	C	M
	Pinned Owner	M	M	M	D	M
	Unowned Central	C	C	C	D	M

Delegate (D). Note that our Delegate AMO implementation can only be applied to the RU state. Thus, we can define a design space of $2^4 \times 3 = 48$ different static policies for the directory. We selected five policies (defined in Table 1) from the 48 possible combinations based on their performance and use cases. We have split the static AMO policies into two groups. The first group consists of static policies that can be implemented with the existing CHI transactions. The second group contains three newly proposed static AMO policies.

All-Central: is the default static policy, executing all AMOs arriving at the directory locally. The All-Central policy has been the default AMO policy for recent CHI-based systems, such as NVIDIA’s Grace and AWS’s Graviton 3 and 4. As discussed previously, All-Central performs well in scenarios where multiple updaters are present, and AMO accesses have low temporal locality.

All-Migrate: always migrates the cache line ownership to the requester. This policy can already be implemented in CHI-based systems by allowing far AMOs in the L1D to send AMOs downstream and then forcing the last level of private cache, typically the L2, to issue ReadUnique requests and perform RMW operations locally when receiving an AMO. All-Migrate behaves similarly to near AMOs, where the core issues a ReadUnique, except that it prevents cache lines from being allocated at the L1. This feature is helpful when AMOs have a large memory footprint and low reuse, as near AMOs may cause L1 to be trashed. All-Migrate does not require an ALU in the directory.

Present Central: is a variation of the All-Central policy in which cache lines not present in the cache hierarchy are directly loaded into the requester’s L2. This can be done efficiently using features such as the direct memory transfer (DMT) [3], which transfers cache lines from the memory controller to the L2 without the directory’s indirection. This behavior is helpful for applications with a large memory footprint that do not fit in cache memory because AMOs missing on the LLC would reduce latency compared to centralized AMOs. Furthermore, for uncontended cases when only one requester accesses a location, it’s helpful to migrate the cache line to its local cache on first access.

Pinned Owner: is a variation of the All-Migrate policy that issues Delegate transaction when a single private cache owns a cache line. This policy assumes that if a private cache owns a cache line, it is probably updating the line, and sending an invalidation would be harmful. Thus, the Pinned Owner policy issues migrating AMOs by default, assuming the new owner would reuse the line. Nevertheless, when an owner already exists, the protocol issues a delegate transaction to avoid disturbing the owner. This protocol benefits CAS workloads, avoiding invalidating the owner copy

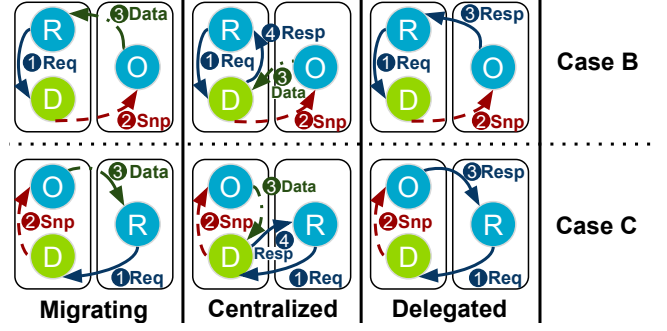


Figure 5: Critical path msgs. between requester (R), directory (D), and owner (O) for migrating, centralized, delegated.

between CAS updates. Like All-Migrate, Pinned Owned does not need the directory’s ALU either.

Unowned Central: is a mixture of the two previous policies that tries to capture the best of both worlds. Unowned Central applies a centralized policy when the cache line is either shared or exclusive to the directory, but not owned by a private cache. If the cache line is not present in any cache, the directory allocates the line to the requester. Otherwise, the directory delegates AMOs to the owner.

4.3 Chiplet-Aware AMO Policy

Applying *centralized*, *delegated*, and *migrating* AMOs to a chiplet-based system results in different tradeoffs. Each message flow behaves differently depending on whether messages traverse a short or a long latency link. This depends on where the requester, the directory, and the owner of the cache line are placed, as already explained in Section 3.1. Figure 5 completes the data flow analysis done in Figure 3 for the other two possible cases (B and C) in a dual-chiplet configuration.

In Case B, the requester and the directory are in the same chiplet, while the owner is remote. Migrating and centralized invalidate the owner crossing the chiplet interconnection two times, one being a line transfer message. However, migrating requires one message less than centralizing, finishing earlier. On the other hand, delegation avoids transfers between chiplets.

In Case C, the owner and the directory are in the same chiplet, and the requester is remote. All three schemes send two messages across the interconnection. However, Migrate is the only transaction that performs a cache line transfer between chiplets. Comparing centralized and delegated AMOs, both cross the interconnection twice, but the centralized transaction has a longer critical path. Thus, the best transaction for this scenario is delegating the AMO.

Out of the three cases, case C has a lower cost for centralized and delegated AMOs because neither transaction performs cache line transfers between chiplets. The main peculiarity of case C is that both the cache line owner and the directory are in the same chiplet. Thus, we conclude that forbidding cache line owners in a different chiplet than the directory reduces cache line transfers between chiplets when multiple requesters exist.

We have designed a Chiplet-Aware policy (Table 2) to account for the requester’s and cache line owner’s locations. This Chiplet-Aware policy extends the Unowned Central policy to prevent transferring a cache line to a different chiplet. The first difference between the Chiplet-Aware policy and Unowned Central is that, on

Table 2: Chiplet-aware policy considering the location of the requester (Req.) and the owner (Own.) w. r. t. the directory.

	UC/RSC/ UD/RSD	RU (Own. Local)	RU (Own. Remote)	I
Req. Local	C	D	C	M
Req. Remote	C	D	C	C

an LLC miss, the Chiplet-Aware policy checks the chiplet location of the requester. If the requester is on the same chiplet, the cache line is allocated to the requester; otherwise, it is allocated to the directory to perform the RMW operation. The second difference is that on an AMO with a cache line owner, the Chiplet-Aware policy checks the cache line owner’s chiplet location. If the owner is in the directory chiplet, it delegates the RMW operation; otherwise, it revokes the cache line ownership to centralize AMOs.

5 Delegato: Delegate Trace Optimization

Next, we introduce Delegated Trace Optimization (Delegato), a mechanism for propagating dynamic cache line reuse information to the directory. We leverage this mechanism to develop an AMO predictor that seamlessly selects the best placement for AMOs.

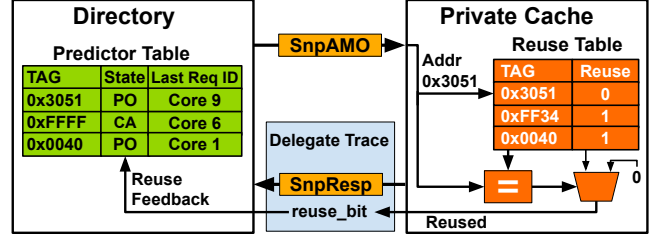
5.1 Information Outage at the Directory

AMO predictors are one type of coherence predictors, which are hardware structures that select the best coherence transaction to optimize single-thread performance or the global throughput [52, 65, 66, 67, 85, 86, 108]. Coherence predictors make informed decisions based on the metadata gathered from the workloads’ access patterns. Metadata also provides feedback for the prediction mechanism about whether a past decision was good or bad. Thus, placing a coherence predictor in the directory seems like an opportunity to enhance the metadata gathering information from all the cores’ private caches. The directory’s available metadata consists of requests from private caches, the owner’s ID, and the list of sharers. Several works propose coherence predictor designs placed in the directory [65, 67, 86, 108].

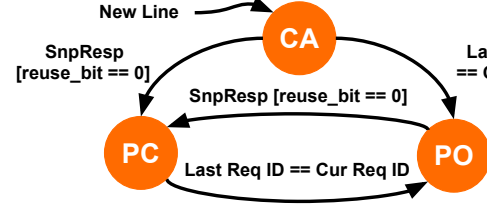
A coherence predictor that aims to predict whether to delegate an AMO needs to know if the owner is taking advantage of holding the line with exclusive permissions. Conventional private caches evict cache lines with low reuse based on heuristics like the least recently used (LRU) chain [54, 114]. However, recent multicore systems are trending towards making private caches bigger to keep the workload’s working set local. For instance, the latest high-performance cores have up to 2 MB L2 cache per core [16, 37]. Such large private caches act as filters for the workload accesses, preventing the LLC from reconstructing the original workload access patterns. The only information that arrives at the LLC is about misses and evictions, not reuse. Therefore, we need a mechanism to transport data locality information to the directory.

5.2 Tracing Reuse

We introduce the Delegate Tracing Optimization (Delegato) to determine if a cache line owned by a private cache is being used. Delegato can effectively communicate information from private



(a) Delegato trace flow between the directory and the private cache.



(b) Predictor Table’s state transitions between Chiplet-Aware (CA), Present Central (PC), and Pinned Owner (PO).

Figure 6: Architecture of Delegato.

caches to the directory, reusing the message flow of Delegate transactions. On an SnpAtomic message, the cache line owner can notify the directory whether the cache line is being reused using a one-bit field (reuse_bit) in the SnpResp message, as shown in Fig 6a. Thus, the directory gets feedback whenever it issues a Delegate transaction, working like a heartbeat. Once private caches no longer reuse a line, the directory is notified in the next Delegate transaction.

Nevertheless, Delegato requires the implementation of a mechanism in the private caches to set the reuse bit on every Delegate request. Such a mechanism could be driven by programmers’ hints like cache management instructions or stashing hints [51]. This way, programmers could pin or unpin cache lines in the private caches, as in the Lock instruction in x86 [5, 53]. Alternatively, private caches can track delegated cache lines’ reuse patterns using low-cost hardware, like in Fig. 6a. Such a mechanism can be integrated with an already existing reuse tracking mechanism, like Re-Reference Interval Prediction (RRIP) [55], used in Neoverse V2 for L2’s cache replacement [16].

5.3 Delegato Predictor

In this section, we describe the architecture of our Delegato-based coherence predictor, depicted in Figure 6. The predictor consists of two tables, one placed in the private cache (i.e., the L2) to track cache line reuse (Reuse Table) and the other in the directory to make predictions (Predictor Table) about AMO placement.

We have drawn inspiration for our design from the Reuse Table in previous reuse-based predictors, such as SHIP [114] and DynAMO [102]. When a core issues an AMO to the L2 and the cache line is in either exclusive state (UC or UD), the cache controller checks the Reuse Table to set the reuse field. If the line is not tracked, it evicts an entry using the LRU heuristic and sets the reuse field to one. When SnpAMO messages arrive in L2, the cache controller checks the Reuse Table to set the reuse_bit of the SnpResponse message. The reuse_bit is set if the line is being tracked and the reuse field of the table is set to 1, or if the cache line is in Unique state in upstream caches. Otherwise, the cache controller responds

Table 3: gem5 configuration.

Processor	
Core count	32 out-of-order cores
Fetch, decode, commit width	8 insts/cycle
Dispatch, issue width	13 insts/cycle
Reorder buffer	224 entries
Load and store queues	76 entries, 58 entries
Cache Memory Hierarchy	
Private L1 I&D caches	64 KiB/core, 4-way, 3-cycle access
Private L2 cache	1 MiB/core, 8-way, 8-cycle access
Shared L3 cache	Mostly-exclusive, 32 slices of 1 MiB, 16 ways, 12-cycle access lat.
Coherence protocol	MOESI-like AMBA 5 CHI specification
Network topology	Each chiplet 6 × 4 2D mesh
Cross-chiplet	450GB/s each direction 50 ns lat.
Main Memory	
Type	DDR5, 8GB storage
Channel	8 channels, 31 GB/s per channel

with 'no-reuse' (0). At the end of the transaction, the reuse field of the Reuse Table is reset. This way, the Reuse Table notifies data reuse only when there has been a local AMO between two Delegate transactions.

The Predictor Table selects the best policy to apply to every AMO in the directory. The directory uses the address of the request to index the Predictor Table. If the cache line is not tracked, a new entry is allocated applying LRU, and the cache controller applies the Chiplet-Aware policy by default. If the TAG is in the table, the cache controller applies the static policy stored in the state field. This field encodes a state machine with three states/policies: Pinned Owner (PO), Present Central (PC) or Chiplet-Aware (CA), shown in Figure 6b. For every AMO, the cache controller stores the last requester ID. If a requester performs two AMOs in a row, the cache controller transfers the cache line ownership to the requester using a Migrate transaction and sets the policy to Pinned Owner. Thus, future updates to the same line from the same requester would be performed locally in its L2.

When an AMO triggers a Delegate transaction, the L2 where the AMO is being delegated replies with reuse information. If the line is being reused, the directory maintains the Pinned Owner policy. Otherwise, the directory changes the policy to Present Central, centralizing the AMOs in the directory, thus recovering the cache line ownership when it is not reused.

6 Evaluation

6.1 Experimental Methodology

Simulation Infrastructure: We have used the gem5 (v23.1.0) performance simulator [75] to evaluate the proposed infrastructure. Gem5 includes cycle-approximate models of various system components, including cores, a cache hierarchy with a detailed mesh-based network-on-chip (NoC), and memory controllers. We use the Arm ISA to model CPUs, Ruby to model the AMBA 5 CHI coherence protocol, and HeteroGarnet [12] to model the mesh NoC. We have used gem5's full system simulation mode to execute the benchmarks on top of an operating system (Ubuntu 22.04 with Linux Kernel 5.4.65). **Baseline System:** We aim to simulate a NUMA multi-chiplet system with two chiplets connected through a memory-coherent interconnect. Each chiplet integrates 16 high-performance out-of-order

Table 4: Benchmark inputs & characteristics.

	Name	Code	Input	Op	Prim	APKI
Classic	Barnes	BAR	16K [113]	CAS	Lock,Bar	1.38
	FMM	FMM	16K [113]	CAS	Lock,Bar	0.30
	Radiosity	RAD	room [113]	CAS	Lock,Bar	2.39
	Raytrace	RAY	car [113]	CAS	Lock,Bar	0.20
	Fluidanimate	FLU	100K [13]	FPA	Lock,Upd	6.60
	Dedup	DUP	media [13]	CAS	Lock	0.10
Graph	Betweenness Centrality	BC	Kron [72]	CAS,SET	Upd	5.69
	Breadth-First Search	BFS	Uni [32]	CAS,SET	Upd	0.65
	SSSP	SSSP	Kron [72]	CAS,MINU	Upd	0.46
Lock-Free	Hash Table	LFH	16k ins/del	CAS	Upd	1.46
	Key-Value	LFK	16k ins/del	CAS	Upd	0.37
	Queue	LFQ	32k ins/del	CAS	Upd	12.40
	Stack	LFS	32k ins/del	CAS	Upd	1.27
	Scalable Circular Queue	SCQ	10 ⁶ ins/del	CAS,FAA	Upd	58.55
	Wait Free Circular Queue	WCQ	10 ⁶ ins/del	CAS,FAA	Upd	38.30
	Wait Free Queue	WFQ	10 ⁶ ins/del	CAS,FAA	Upd	14.26
	Histogram	HIS	Hateren[107]	FAA	Upd	74.71
	Lulesh	LUL	100x100x100	FPA	Bar,Upd	74.71
	Kmer Count	KME	Ecoli [27]	FAA	Upd	7.64
	Radix Sort	RSO	4 MB vect.	FAA	Bar,Upd	52.37
	SPMV-CSC	SPC	JP [27]	FPA	Upd	46.61
	SPMV-COO	SPO	JP [27]	FPA	Upd	54.62

cores. All CPUs run at 3 GHz, while the NoC and the LLC run at 2 GHz. Both chiplets are interconnected through a high-bandwidth interconnect, serving up to 450 GB/s in each direction and with a latency of 50 ns. We implement two AMO policies (near and far) for the L1 following Neoverse N2 manual [2]. Near AMO policy enforces all AMOs to be performed at the L1, while the far AMO issues an AMO when the line is not present in the L1 with exclusive permissions. Table 3 summarizes microarchitectural details.

Workloads: We use a representative set of parallel applications:

- **Classic Parallel Benchmarks:** We evaluate benchmarks from classic parallel benchmark suites such as Splash-4 [44] and PARSECS [20]. We have discarded those applications under 0.1 AMOs per kilo instruction (APKI).
- **Graph Analytics:** We selected three graph kernels containing AMOs: betweenness centrality, breadth-first search, and single-source shortest path (SSSP) from the GAP [10].
- **Lock-Free Data Structures:** We evaluate a lock-free hash table, queue, and stack [60]. Additionally, we test state-of-the-art lock-free circular queue implementations [90, 91, 115].
- **Other Parallel Kernels:** We have evaluated two scientific applications from the genomic-domain (Kmer-count [77]), and Shock Hydrodynamics (LULESH [73, 103]). We have used the SPMV kernels for CSC and COO matrix formats from a well-known sparse deep learning library [100, 110]. We include the classic Radix Sort [101] and Histogram [15].

Table 4 details the acronym, the inputs, the type of AMO used (Op), the synchronization primitive using the AMO (Prim), and the APKI ratio for each evaluated application. The types of atomic operations can be compare-and-swap (CAS), integer fetch-and-add (FAA), floating-point addition (FPA), bitwise OR (SET), and integer unsigned minimum (MINU). Prim can be 'Lock' for mutexes, 'Bar' for barriers, and 'Upd' for direct memory updates.

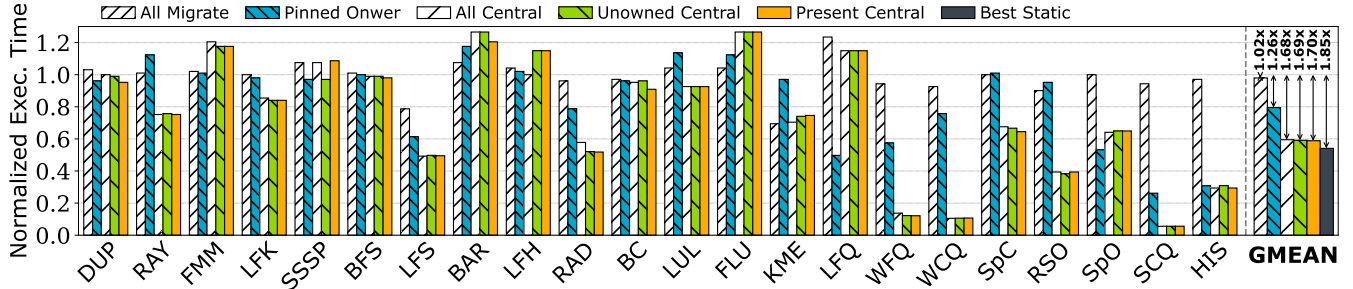


Figure 7: Normalized execution time of five static AMO policies with respect to Near AMOs. The rightmost bars are the Geomean normalized execution time with speed-up labels.

6.2 Evaluation of Static Directory Policies

Figure 7 shows the normalized execution time of the different static AMO policies (Section 4.2) normalized with respect to near AMOs. We have selected near AMOs as the reference normalization point because centralized AMOs are not always the best-performing scheme, and most systems only perform near AMOs [4, 50]. The existing policies are depicted using white bars, while the proposed policies use colored bars. The right part of the figure represents the geometric average execution time of each policy (GMEAN). The benefits of applying each of the static AMO policies are strongly application-dependent. We can divide policies into three groups based on their behavior.

The first group comprises only the All Migrate policy, the closest policy to near AMOs. All Migrate potential gains are limited since AMOs are executed on the requester’s L2 rather than the L1. Thus, this policy only helps when AMO-targeted lines evict useful lines allocated in the L1, achieving the best speedup on KME (1.44×), where AMOs are performed over a vast data set with temporal locality. All Migrate achieves on average a 1.02× speedup.

The Pinned Owner policy forms the second group. The Pinned Owner achieves modest speed-ups compared to centralized policies, achieving an average of 1.26× speedup across all applications. Like All Migrate, the Pinned Owner policy does not require placing an ALU in the directory, but it outperforms the former policy in most benchmarks. The Pinned Owner policy performs well in CAS-based applications like LFS, LFQ, SCQ, and HIS. Thus, we can say that the Pinned Owner policy is CAS-affine. This CAS affinity makes Pinned Owner the best-performing policy in LFQ.

The third group comprises All Central, Unowned Central, and Present Central policies, as they perform similarly for most applications. The three policies make the same decision for most of the cache states, favoring the execution of AMOs in the directory. However, this behavior causes all three policies to downgrade the system performance in FMM, SSSP, BAR, LFH, FLU, and LFQ. All these applications use CAS-based mutex or algorithms, so centralizing AMOs in the directory induces CAS RMW to fail. We name this behavior CAS-disruptive. On the other hand, all three policies speed-up FAA-based applications, such as WFQ, WCQ, SPC, SPO, SCQ, and HIST, as centralizing RMWs increases AMO throughput. On average, All Central, Unowned Central, and Present Central speed-up applications by 1.68×, 1.69×, and 1.70×, respectively.

Finally, the Best Static bar on the geomean represents the performance the programmer can achieve by selecting the best static AMO policy for each workload. This method achieves an average

1.85× speedup over near AMOs and 1.10× over All Central policy. However, profiling applications and configuring the system temporarily [50] to use the best static policy is impractical for most use cases, thus motivating mechanisms to select policies dynamically.

6.3 CAS Affinity

As shown in Section 6.2, policies have different affinities towards CAS operations. To further understand the impact of the proposed policies on CAS operations, we use a microbenchmark where all cores update a shared counter using CAS instructions. Figure 8 shows the speedup achieved by each static policy with respect to near AMOs. Additionally, we have plotted the access miss rate of AMOs in the private caches (black dots). We observe that CAS misses are inversely proportional to speedup in this CAS microbenchmark. The All Migrate policy slightly degrades performance, as cache lines are fetched at the L2 instead of the L1, increasing CAS’s latency and the probability of failure. For All Central, Unowned Central, and Present Central policies, every CAS operation is performed in the directory, reducing overall system throughput, as remotely executed CAS operations are more likely to fail.

On the other hand, the Pinned Owner policy substantially reduces the benchmark execution time, achieving a 1.7× speedup with respect to near AMOs. The Pinned Owner has the lowest ratio of L2 misses from CAS AMOs, thereby reducing the CAS failure rate and increasing overall system throughput. The pinned owner removes invalidations from other cores while the owner updates the cache line, effectively reducing data movement.

This experiment demonstrates why centralizing AMOs in the directory negatively impacts the performance of CAS AMOs and CAS-based locks (e.g., mutexes) under high contention scenarios. However, centralizing updates can be beneficial in cases with a lower level of contention (e.g., LFS) as updaters do not reuse the cache line and the probability of failing a CAS operation is lower due to the lower contention.

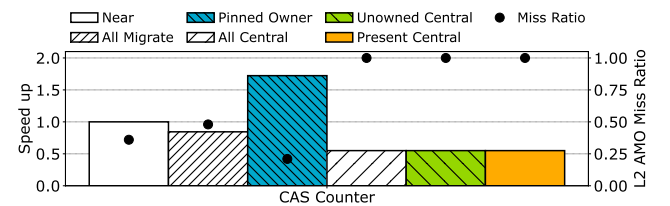


Figure 8: Execution speedup of CAS Counter for static AMO policies normalized with respect to near AMOs. Black dots are the ratio of AMOs that miss on the private caches.

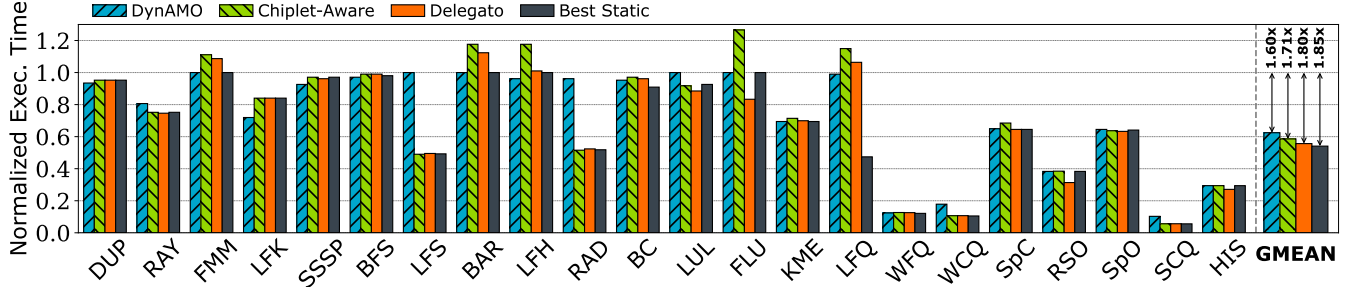


Figure 9: Normalized execution time of dynamic AMO policies with respect to Near AMOs. The rightmost bars are the Geomean normalized execution time with speed-up labels.

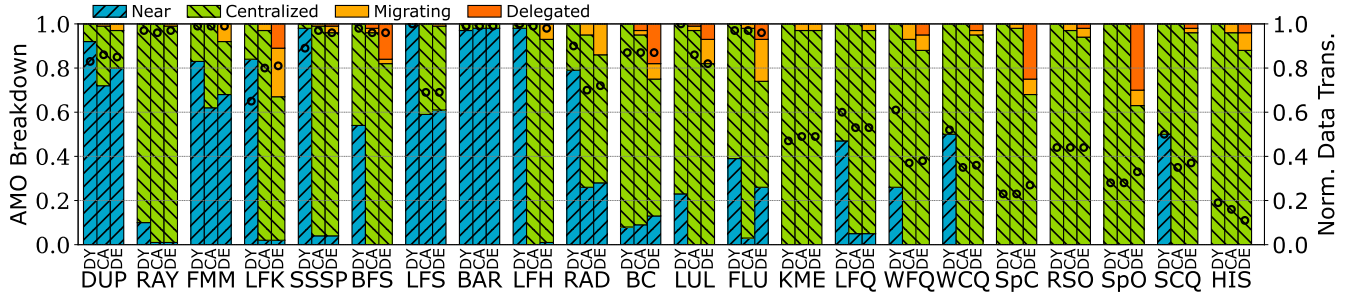


Figure 10: AMO type breakdown (left-y-axis) and normalized cache line transfer with respect to near AMOs (right-y-axis) for DynAMO, Chiplet-Aware, and Delegato schemes.

6.4 Evaluation of Dynamic Policies

This section compares the performance of the Chiplet-Aware AMO policy, the state-of-the-art AMO predictor DynAMO [102], and our proposed Delegato predictor. Figure 9 shows the normalized execution time of each mechanism for all workloads. We plot the best static policy (Best Static) in every benchmark for reference.

DynAMO is the state-of-the-art coherence predictor for AMOs. While the Delegato predictor selects where the AMO would be executed once its request arrives at the directory, DynAMO selects whether the L1 must issue a Read Unique or an AMO transaction to downstream cache levels. These AMO transactions are executed centrally in the directory, as current systems do not support Delegate or Migrate transactions. The DynAMO predictor has a geometric average speedup of 1.60 $\times$ over near AMOs. DynAMO has a lower speedup than centralized AMOs, as it seldom issues far AMOs due to its conservative design. DynAMO only issues an AMO transaction when the cache line is invalid in the L1D, and the predictor tracks that the cache line has not been reused in the past. Nonetheless, DynAMO outperforms the best static policy in DUP, LFK, SSSP, and LFH, achieving speedups of 1.07 $\times$, 1.39 $\times$, 1.08 $\times$, and 1.04 $\times$, respectively. Its low average speedup is caused by the low performance in LFS, RAD, WCQ, and SCQ applications performing near AMOs rather than centralized AMOs.

The Chiplet-Aware policy outperforms any other static policy, achieving an average speedup of 1.71 $\times$ over near AMOs without requiring any area overhead. As this policy is based on the Unowned Central policy, it slows down the FMM, BAR, LFH, FLU, and LFQ applications. Nevertheless, the chiplet location information helps improve the performance with respect to Unowned Central for DUP, BAR, KME, and HIST.

Delegato provides the best overall performance gains over the baseline (near AMOs) with a geometric mean of 1.80 $\times$. Thanks to its dynamic policy selection, Delegato outperforms the best static policies in RAY, SSSP, LUL, FLU, RSO, and HIS. By upgrading frequent updaters through a Migrate transaction, the Delegato predictor can efficiently reduce requests to the directory and improve AMO latency. However, such optimization can harm the application’s performance when upgrading an infrequently accessed cache line. In turn, the tracing mechanism helps the directory regain control of the cache line when the owner is not updating it to restrict performance degradation. However, the Delegato predictor degrades the performance of FMM, BAR, and LFQ applications compared to near AMOs. The predictor can do little in those cases because near AMOs are better suited. Therefore, combining Delegato with an L1 predictor like DynAMO can fix the performance degradation. Furthermore, the Reuse Table can be combined with the AMO Metadata Table from DynAMO to save space. However, combining both predictors would harm Delegato’s performance in other benchmarks such as RAY, LFS, RAD, WCQ, and SCQ due to its conservative AMO policy. Future L1 AMO predictors improving DynAMO accuracy, combined with Delegato, could further increase the overall system performance.

6.5 AMO Breakdown and Data Movement

Figure 10 details the portion of each AMO transaction type executed by DynAMO (DY), Chiplet-Aware (CA), and Delegato (DE). Additionally, for each scheme, we have computed the number of cache lines transferred across the NoC normalized with respect to the baseline (performing only near AMOs), shown with a circle marker (secondary y-axis).

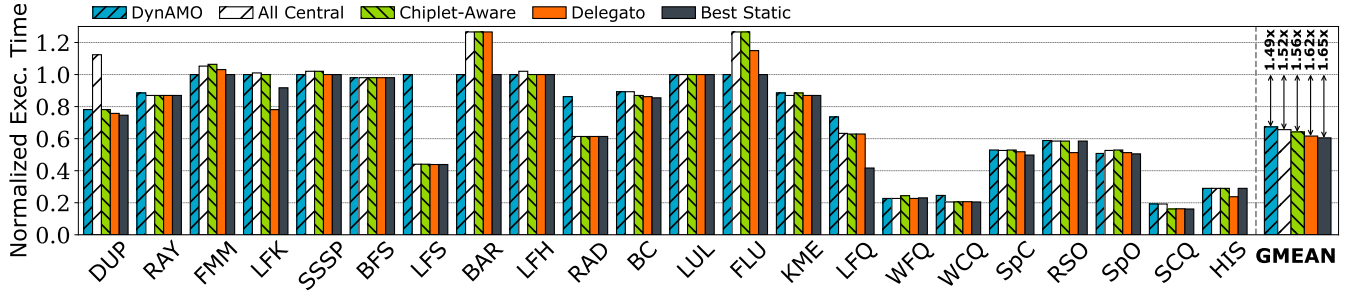


Figure 11: Normalized execution time of five AMO placement mechanisms with respect to Near AMOs in a UCIE-based system. The rightmost bars are the Geomean normalized execution time with speed-up labels.

We can observe that DynAMO only executes near and centralized AMOs. Although DynAMO is the scheme with the highest ratio of near AMOs, its predictor is capable of identifying benchmarks with low data locality, such as SpC, RSO, SpO, and HIS, to issue centralized AMOs. DynAMO is capable of reducing data movement by only fetching those cache lines that have reuse potential, achieving the highest data movement reduction in DUP, LFK, and KME.

Chiplet-Aware performs mostly centralized AMOs, only performing near the block when it is in the Unique state at the L1D. However, this scheme can perform migrating AMOs when the block is not present in any cache; this is important in LFK, RAD, BC, LUL, KME, WFQ, SpC, RSO, and HIS. The number of migrating AMOs is low, as this transaction can only be performed if the requester belongs to the same NUMA domain as the directory. Although Chiplet-Aware can perform delegated AMOs when the cache line is held exclusively by a single owner, the low number of owners makes the ratio of delegated AMOs negligible, only noticeable in BFS, BC, and LUL. Although Chiplet-Aware minimizes the data transfer of RAY, BFS, LFS, RAD, LFQ, WCQ, SpC, SCQ, and SpO, this data movement reduction does not always translate into speed-ups, as some AMO transactions suffer from higher latency.

Comparing Delegato against the Chiplet-Aware policy, we can observe that Delegato significantly outperforms in terms of migrating AMOs in DUP, FMM, LFK, SSSP, LFS, LFH, RAD, BC, LUL, FLU, WCQ, SpC, SpO, and HIS. The Predictor Table detects consecutive accesses from the same requester and migrates the cache line to the requester. Then, the Pinned Owner policy is applied, increasing the ratio of delegated AMOs in LFK, SSSP, BFS, BC, LUL, FLU, WFQ, WCQ, SpC, SpO, SCQ, and HIS. As a result, we can observe how the ratio of near AMOs increases in FMM, LFS, RAD, and FLU, as AMOs hit in the L1D. Despite Delegato sacrificing some data movement reduction to improve the overall latency of AMOs, it achieves the highest data movement reduction in LUL, FLU, RSO, and HIS.

6.6 Low Latency Interposer

The Universal Chiplet Interconnect Express (UCIE) promises high bandwidth, power-efficient, and low-latency (2 ns) on-package connectivity between chiplets [26]. Thus, UCIE can reduce the latency penalty of transferring cache lines between chiplets and change the tradeoffs of the different policies. We have re-executed our experimental setup with an interconnection latency of two nanoseconds. Figure 11 shows the normalized execution time for five selected

AMO mechanisms with respect to near AMOs. We have selected All Central and Pinned Owner policies because they are the most representative of static policies. Regarding dynamic policies, the Chiplet-Aware policy, the DynAMO coherence predictor, and the Delegato predictor are shown. The 2-ns interconnection reduces the potential gains of all schemes with respect to near AMOs compared to a 50-ns interconnection. The shorter latency reduces the cost of transferring cache lines between chiplets, favoring near AMOs.

The DynAMO predictor behaves similarly to the previous experiment, having an average speed-up of 1.49× across all benchmarks. DynAMO does not utilize centralized AMOs for LFS, RAD, and LFQ applications, resulting in diminished performance. Furthermore, DynAMO does not perform best for any benchmark.

All Central and Chiplet-Aware policies achieve similar speed-ups in most applications. On average, both policies have 1.52× and 1.56× speed-ups, respectively. Both policies keep considerable speed-ups in LFS, RAD, WFQ, WCQ, SpC, RSO, SpO, SCQ, and HIS, although most speed-ups are lower than with a 50-ns interconnection. Although some speed-ups disappear in LFK and LUL, both policies improve their performance in LFQ. Since the Chiplet-Aware policy assumes a long latency interconnection between chiplets, when this latency reduces, the benefits of applying this policy diminish and match the performance of All Central. The only difference between the two policies is in the DUP, where allocating cache lines that miss on the LLC in the requester reduces the number of accesses to the directory and speeds-up the application by 1.28×.

The Delegato predictor is the hardware scheme with the best performance, with a geometric mean of 1.62×. Delegato outperforms the best static policy in LFK, RSO, and HIS applications. Again, these speed-ups are achieved through the upgrade and trace mechanism. Although the speed-ups achieved in LUL and FLU with a 50-ns interconnection vanish, on average Delegato outperforms the All Central policy by 1.07×. The Best Static policy is only 1.02× faster than Delegato, achieving an average speed-up of 1.65×. Thus, proving the robustness of the Delegato predictor.

6.7 Delegato Predictor Sizing

We have performed a design space exploration for the Delegato predictor, shown in Figure 12. Every experiment changes one parameter from the predictor and fixes the other parameters for the best-performing configuration. The best configuration uses 128

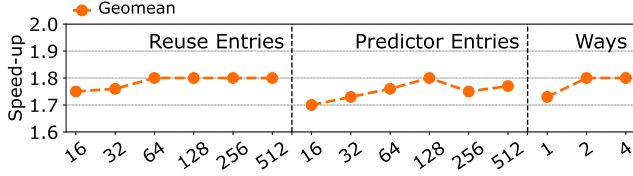


Figure 12: Execution speedup of CAS Counter microkernel for different static AMO policies normalized with respect to near AMOs. Black data points represent the ratio of AMOs that miss on the private caches.

entries for the Reuse and the Predictor tables, using 2-way associativity for both structures.

The first experiment shows the performance of the Delegato predictor as the number of entries of the Reuse Table changes. This table tracks the reuse information for AMO-accessed cache lines at the L2. The Reuse table hits the best performance with a small number of entries (about 64). The reason is that few lines have actual reuse at the L2. Therefore, for bigger sizes, most tracked lines have zero reuse.

Regarding the sizing of the Predictor Table, we can observe that increasing the number of tracked lines increases the predictor speed-up up to 128 entries. However, the performance of 256 entries is lower because the extra tracked cache lines lead to mispredicted upgrade transitions. The reason is that long-lasting entries in the predictor can trigger the update mechanism when a core requests two AMOs for the same address. Despite those lines having no reuse, the predictor detects an upgrade opportunity.

Finally, we test three different associativity configurations for the Reuse and the Predictor Table. Increasing the number of ways from 1 to 2 reduces the number of conflicts in both tables. However, due to the low number of cache lines accessed by AMOs, increasing further the number of ways does not improve the performance.

6.8 Area Overhead and Complexity

Each Reuse Table entry requires 50 bits, 49 for the physical address TAG and one for the reuse field. Hence, a 128-entry Reuse Table requires 800 bytes. Following the same methodology as in Section 3.3, we have computed the area of the Reuse Table. On a 7 nm process running at 3GHz, each Reuse Table occupies $743 \mu m^2$, totaling $0.024 mm^2$ for a 32-core design. Secondly, the Predictor table entries require 56 bits, comprising 49 bits for the physical address TAG, 5 bits for the requester ID, and 2 bits for the policy field. A Predictor Table of 128 entries would require 896 bytes, using $807 \mu m^2$. Replicating the predictor table on each LLC slice would total $0.026 mm^2$.

Adding the reuse_bit field in SnpResp from the AMBA CHI protocol is the most expensive component of the Delegato design, as it would require adding one bit for every response message. Opportunely, response messages have the DataPull field, which Stashing transactions use to allow or disallow data installation in the private caches. Thus, reusing that field allows the implementation of Delegato without cost.

Delegato requires performing RMW operations at the L2. Thus, we must account for an additional $2894 \mu m^2$ for a floating-point-capable ALU at the L2 level. Note that an alternative implementation would be to share the ALU with the L1D cache.

To put the total Delegato area cost into perspective, one Neoverse V1 with L1 and L2 caches has an area of $2.52 mm^2$, so a 32-core system would use $80.64 mm^2$. Implementing Delegato in such a system would require adding $0.142208 mm^2$, which includes the predictor and the L2 FP capable ALU, representing 0.17% of the total area. Therefore, we can conclude that Delegato is a cost-effective coherence predictor.

7 Related Work

Academic literature has observed that AMOs are inefficient in conventional cache hierarchies and has profusely explored hardware and software techniques to enhance their execution.

Efficient Synchronization Primitives: There is a long history of efficient primitives such as lightweight locks, read-copy-update mechanisms, and lock-free memory allocators [6, 59, 81, 82].

Near AMOs: Most modern computer microarchitectures support near AMOs [35, 50, 53]. Recent academic works have covered how near AMO can be executed more efficiently [4, 38, 64, 66], either by performing multiple AMOs [38, 64], reducing their serialization cost [4], or efficiently implementing RMW operations [66, 121].

Centralized AMOs: Some classic commercial systems [41, 61, 69, 99] were the first to implement centralized AMOs. Despite modern systems supporting different centralized AMOs [48, 70, 94], there are few academic works covering them [66, 102, 120, 121]. ATUN [66] is an efficient open-source atomic unit capable of executing centralized AMOs for RISC-V. Atomic Cache [121] is an in-cache computing design to accelerate RMW operations in RISC-V general-purpose graphics processing units (GPGUs).

Coalescing Optimizations: Several academic works exploit data commutativity to coalesce atomic updates [7, 8, 24, 29, 36, 96, 119]. These approaches usually change the coherence protocol [119], or the cache hierarchy [7, 8, 24, 29]. Their main drawback is that they depend on data locality to coalesce AMOs. Their applicability is orthogonal to delegated or migrating AMOs and the Delegato mechanism.

Interconnection Optimizations: Although chiplet designs have emerged recently, NUMA systems have existed for a long time. Several works cover the problem of maintaining cache coherence of multiple nodes interconnected through a long-latency link [17, 25, 39, 76, 83, 84, 89, 104]. Although NUMA coherence overhead is a long-standing challenge in computer architecture, academia continues to make significant progress in understanding and mitigating its effects. A recent work [25] proposes a multi-chiplet GPU architecture to reduce remote coherence interconnect traffic.

Coherence Misses Optimizations: A plethora of literature covers cache coherence optimizations to mitigate or remove coherence misses. The two main mechanisms to remove coherence misses are the write-update coherence protocols and the write-push primitive. Write-update coherence protocols were proposed in the 80s as an alternative to reduce data access latency in highly shared data programs [80, 105]. Write-update protocols forward write data to sharers so they can update their local copies, thus dramatically decreasing coherence misses. However, modern computers do not implement write-update protocols due to the high interconnect traffic they generate.

Alternatively, the write-push primitive inherits the idea of write-update protocols to propagate write data, enabling programmers to send data explicitly from producers to consumers' caches [1, 40, 63, 71, 95, 97, 98]. The AMBA CHI protocol features the write-push primitive through the WriteUniqueStash transaction [48]. Unlike Delegate transactions, WriteUniqueStash transactions require the target cache to be indicated where the new data should be installed and cannot be applied to AMOs. Alternatively, issuing a centralized AMO followed by a StashOnce to install the newly updated cache line in a target cache (as proposed in [120]) can reduce coherence misses but not the data movement.

Data Location and Coherence Prediction: In the past, several works have proposed microarchitectural prediction to estimate the best placement for shared data [78, 86, 102] or the best coherence state [52, 65, 65, 67, 85, 86, 108]. STAP is a coherence predictor that dynamically pushes shared data depending on the communication pattern [86]. SpecLock uses coherence hints to predict L1 memory accesses for fast lock forwarding [78].

8 Conclusions

This work proposes and implements two novel types of far AMOs: delegated and migrating. Thus, we extend modern computer systems' remote AMOs beyond the current centralized execution at the coherence point (i.e., the directory). Integrating all three types of far AMOs enables the directory to select the most suitable location to execute AMOs in a system.

We analyze the implications of supporting all transaction types in the directory design space and propose four AMO policies for selecting the appropriate transaction based on cache line state, ownership, and location in a multi-chiplet system. Our evaluation of these AMO policies shows that: (i) one of our proposed policies achieves a notable speed-up despite not performing RMW operations at the directory; and (ii) our proposed Chiplet-Aware static policy outperforms current far AMO implementations.

Moreover, we propose a Delegato, a tracing mechanism for delegated AMOs that effectively transports dynamic reuse information from the private caches to the directory. We show Delegato's utility by designing an AMO predictor that seamlessly selects the best placement for far AMOs. The delegato-based predictor achieves a 1.13 $\times$ speed-up over the state-of-the-art AMO predictor and 1.07 $\times$ over the modern far AMO implementation.

Acknowledgments

This research was supported by the Spanish Ministry of Science and Innovation through contracts [PID2023-146511NB-I00], and [PID2022-136454NB-C22] and by the Ministry for Digital Transformation and Public Service, within the framework of the Recovery, Transformation and Resilience Plan - NextGenerationEU [REGAGE22e00058408992]; the Generalitat of Catalunya through contract [2021-SGR-00763]; the Government of Aragón [T58_23R]; the Arm-BSC Center of Excellence, and the European Processor Initiative (EPI) which is part of the European Union's Horizon 2020 research and innovation program under grant agreement No. 800928 and No. 101036168. V. Soria-Pardos has been supported through an FPU fellowship [FPU20-02132]; A. Armejch is a Serra Hunter Fellow.

References

- [1] H. Abdel-Shafi, J. Hall, S.V. Adve, and V.S. Adve. 1997. An evaluation of fine-grain producer-initiated communication in cache-coherent multiprocessors. In *Proceedings Third International Symposium on High-Performance Computer Architecture*. 204–215. <https://doi.org/10.1109/HPCA.1997.569661>
- [2] Arm holdings. 2020. Arm Neoverse N2 Core Technical Reference Manual. <https://developer.arm.com/documentation/102099/0001/The-Neoverse-N2--core>. [Online; accessed 19-June-2025].
- [3] Arm Holdings. 2021. AMBA 5 CHI Architecture Specification. <https://developer.arm.com/architectures/system-architectures/amba/amba-5>. [Online; accessed 19-June-2025].
- [4] Ashkan Asgharzadeh, Juan M. Cebrian, Arthur Perais, Stefanos Kaxiras, and Alberto Ros. 2022. Free atomics: hardware atomic operations without fences. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) (ISCA '22). Association for Computing Machinery, New York, NY, USA, 14–26. <https://doi.org/10.1145/3470496.3527385>
- [5] Ashkan Asgharzadeh, Eduardo José Gómez-Hernández, Juan M. Cebrian, Stefanos Kaxiras, and Alberto Ros. 2024. Hardware Cache Locking for All Memory Updates. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. 566–574. <https://doi.org/10.1109/ICCD63220.2024.00092>
- [6] David F. Bacon, Ravi Konuru, Chet Murthy, and Mauricio Serrano. 1998. Thin locks: featherweight synchronization for Java. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation* (Montreal, Quebec, Canada) (PLDI '98). Association for Computing Machinery, New York, NY, USA, 258–268. <https://doi.org/10.1145/277650.277734>
- [7] Vignesh Balaji and Brandon Lucia. 2022. Improving Locality of Irregular Updates with Hardware Assisted Propagation Blocking. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 543–557. <https://doi.org/10.1109/HPCA53966.2022.00047>
- [8] Vignesh Balaji, Dhruva Tirumala, and Brandon Lucia. 2017. Flexible Support for Fast Parallel Commutative Updates. arXiv:1709.09491 [cs.DC] <https://arxiv.org/abs/1709.09491>
- [9] Scott Beamer, Krste Asanovic, and David Patterson. 2015. Locality Exists in Graph Processing: Workload Characterization on an Ivy Bridge Server. In *2015 IEEE International Symposium on Workload Characterization*. 56–65. <https://doi.org/10.1109/IISWC.2015.12>
- [10] Scott Beamer, Krste Asanovic, and David Patterson. 2017. The GAP Benchmark Suite. arXiv:1508.03619 [cs.DC] <https://arxiv.org/abs/1508.03619>
- [11] Scott Beamer, Krste Asanovic, and David Patterson. 2017. Reducing Pagerank Communication via Propagation Blocking. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 820–831. <https://doi.org/10.1109/IPDPS.2017.112>
- [12] Srikant Bharadwaj, Jieming Yin, Bradford Beckmann, and Tushar Krishna. 2020. Kite: A Family of Heterogeneous Interposer Topologies Enabled via Accurate Interconnect Modeling. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1109/DAC18072.2020.9218539>
- [13] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques* (Toronto, Ontario, Canada) (PACT '08). Association for Computing Machinery, New York, NY, USA, 72–81. <https://doi.org/10.1145/1454115.1454128>
- [14] Bill Blume, Rudolf Eigenmann, Keith Faigin, John Grout, Jay Hoeflinger, David Padua, Paul Petersen, Bill Pottenger, Lawrence Rauchwerger, Peng Tu, et al. 1994. Polaris: The next generation in parallelizing compilers. In *Proceedings of the Seventh Workshop on Languages and Compilers for Parallel Computing*. Citeseer, 141–154.
- [15] Gary Bradski and Adrian Kaehler. 2008. *Learning OpenCV: Computervision with the OpenCV library*. O'Reilly.
- [16] Magnus Bruce. 2023. Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for Next-Generation Cloud Computing, ML and HPC Workloads. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. 1–25. <https://doi.org/10.1109/HCS59251.2023.10254718>
- [17] Paul Caheny, Lluc Alvarez, Said Derradji, Mateo Valero, Miquel Moretó, and Marc Casas. 2018. Reducing Cache Coherence Traffic with a NUMA-Aware Runtime Approach. *IEEE Transactions on Parallel and Distributed Systems* 29, 5 (2018), 1174–1187. <https://doi.org/10.1109/TPDS.2017.2787123>
- [18] Irina Calciu, Dave Dice, Tim Harris, Maurice Herlihy, Alex Kogan, Virendra Marathe, and Mark Moir. 2013. Message Passing or Shared Memory: Evaluating the Delegation Abstraction for Multicores. In *Principles of Distributed Systems*, Roberto Baldoni, Nicolas Nisse, and Maarten van Steen (Eds.). Springer International Publishing, Cham, 83–97.
- [19] Irina Calciu, Justin Gottschlich, and Maurice Herlihy. 2013. Using Elimination and Delegation to Implement a Scalable NUMA-Friendly Stack. In *5th USENIX Workshop on Hot Topics in Parallelism (HotPar 13)*. USENIX Association, San Jose, CA. <https://www.usenix.org/conference/hotpar13/workshop-program/presentation/calciu>

- [20] Dimitrios Chasapis, Marc Casas, Miquel Moretó, Raul Vidal, Eduard Ayguadé, Jesús Labarta, and Mateo Valero. 2015. PARSECS: Evaluating the Impact of Task Parallelism in the PARSEC Benchmark Suite. *ACM Trans. Archit. Code Optim.* 12, 4, Article 41 (Dec. 2015), 22 pages. <https://doi.org/10.1145/2829952>
- [21] Jan Ciesko, Javier Bueno, Nikola Puzovic, Alex Ramirez, Rosa M. Badia, and Jesús Labarta. 2013. Programmable and Scalable Reductions on Clusters. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. 560–568. <https://doi.org/10.1109/IPDPS.2013.63>
- [22] Jan Ciesko, Sergi Mateo, Xavier Teruel, Vicenç Beltran, Xavier Martorell, and Jesús Labarta. 2015. Boosting Irregular Array Reductions through In-lined Block-ordering on fast processors. In *2015 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–6. <https://doi.org/10.1109/HPEC.2015.7322443>
- [23] Jan Ciesko, Sergi Mateo, Xavier Teruel, Xavier Martorell, Eduard Ayguadé, and Jesús Labarta. 2016. Supporting Adaptive Privatization Techniques for Irregular Array Reductions in Task-Parallel Programming Models. In *OpenMP: Memory, Devices, and Tasks*, Naoya Maruyama, Bronis R. de Supinski, and Mohamed Wahib (Eds.). Springer International Publishing, Cham, 336–349.
- [24] Preyesh Dalmia, Rohan Mahapatra, and Matthew D. Sinclair. 2022. Only Buffer When You Need To: Reducing On-chip GPU Traffic with Reconfigurable Local Atomic Buffers. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 676–691. <https://doi.org/10.1109/HPCA53966.2022.00056>
- [25] Preyesh Dalmia, Rajesh Shashi Kumar, and Matthew D. Sinclair. 2024. CPElide: Efficient Multi-Chiplet GPU Implicit Synchronization. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 700–717. <https://doi.org/10.1109/MICRO61859.2024.00058>
- [26] Debendra Das Sharma, Gerald Pasdast, Zhiguo Qian, and Kemal Aygun. 2022. Universal Chiplet Interconnect Express (UCIe): An Open Industry Standard for Innovations With Chiplets at Package Level. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 12, 9 (2022), 1423–1431. <https://doi.org/10.1109/TCPMT.2022.3207195>
- [27] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (dec 2011), 25 pages.
- [28] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (jan 2008), 107–113. <https://doi.org/10.1145/1327452.1327492>
- [29] Vladimir Dimić, Miquel Moretó, Marc Casas, Jan Ciesko, and Mateo Valero. 2020. RICH: implementing reductions in the cache hierarchy. In *Proceedings of the 34th ACM International Conference on Supercomputing (Barcelona, Spain) (ICS '20)*. Association for Computing Machinery, New York, NY, USA, Article 16, 13 pages. <https://doi.org/10.1145/3392717.3392736>
- [30] M. Dubois, J. Skeppstedt, and P. Stenstrom. 1995. Essential Misses and Data Traffic in Coherence Protocols. *J. Parallel and Distrib. Comput.* 29, 2 (1995), 108–125. <https://doi.org/10.1006/jpdc.1995.1112>
- [31] T.v. Eicken, D.E. Culler, S.C. Goldstein, and K.E. Schausser. 1992. Active Messages: A Mechanism for Integrated Communication and Computation. In *[1992] Proceedings the 19th Annual International Symposium on Computer Architecture*. 256–266. <https://doi.org/10.1109/ISCA.1992.753322>
- [32] P ERdS and A R&wi. 1959. On random graphs I. *Publ. math. debrecen* 6, 290–297 (1959), 18.
- [33] Pouya Fotouhi, Sebastian Werner, Jason Lowe-Power, and S. J. Ben Yoo. 2019. Enabling scalable chiplet-based uniform memory architectures with silicon photonics. In *Proceedings of the International Symposium on Memory Systems (Washington, District of Columbia, USA) (MEMSYS '19)*. Association for Computing Machinery, New York, NY, USA, 222–334. <https://doi.org/10.1145/3357526.3357564>
- [34] Matteo Frigo, Pablo Halpern, Charles E. Leiserson, and Stephen Lewin-Berlin. 2009. Reducers and other Cilk++ hyperobjects. In *Proceedings of the Twenty-First Annual Symposium on Parallelism in Algorithms and Architectures (Calgary, AB, Canada) (SPAA '09)*. Association for Computing Machinery, New York, NY, USA, 79–90. <https://doi.org/10.1145/1583991.1584017>
- [35] César Fuguet. 2023. HPDcache: Open-Source High-Performance L1 Data Cache for RISC-V Cores. In *Proceedings of the 20th ACM International Conference on Computing Frontiers (Bologna, Italy) (CF '23)*. Association for Computing Machinery, New York, NY, USA, 377–378. <https://doi.org/10.1145/3587135.3591413>
- [36] M.J. Garzaran, M. Prvulovic, Ye Zhang, A. Julia, Hao Yu, L. Rauchwerger, and J. Torrellas. 2001. Architectural support for parallel reductions in scalable shared-memory multiprocessors. In *Proceedings 2001 International Conference on Parallel Architectures and Compilation Techniques*. 243–254. <https://doi.org/10.1109/PACT.2001.953304>
- [37] Chris Gianos. 2023. Architecting for Flexibility and Value with Next Gen Intel® Xeon® Processors. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. 1–15. <https://doi.org/10.1109/HCS59251.2023.10254694>
- [38] Eduardo José Gómez-Hernández, Juan M. Cebrian, Rubén Titos-Gil, Stefanos Kaxiras, and Alberto Ros. 2021. Efficient, Distributed, and Non-Speculative Multi-Address Atomic Operations. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 337–349. <https://doi.org/10.1145/3466752.3480073>
- [39] JR Goodman and HHJ Hum. 2004. Mesif: A two-hop cache coherency protocol for point-to-point interconnects (2009). URL: <https://www.cs.auckland.ac.nz/goodman/TechnicalReports/MESIF-2009.pdf> (2004).
- [40] James R. Goodman, Mary K. Vernon, and Philip J. Woest. 1989. Efficient synchronization primitives for large-scale cache-coherent multiprocessors. *SIGARCH Comput. Archit. News* 17, 2 (April 1989), 64–75. <https://doi.org/10.1145/68182.68188>
- [41] Allan Gottlieb, Ralph Grishman, Clyde P. Kruskal, Kevin P. McAuliffe, Larry Rudolph, and Marc Snir. 1998. The NYU Ultracomputer—Designing a MIMD, Shared-Memory Parallel Machine. In *25 Years of the International Symposia on Computer Architecture (Selected Papers) (Barcelona, Spain) (ISCA '98)*. Association for Computing Machinery, New York, NY, USA, 239–254. <https://doi.org/10.1145/285930.285983>
- [42] E. Gutiérrez, O. Plata, and E. L. Zapata. 1999. On Automatic Parallelization of Irregular Reductions on Scalable Shared Memory Systems. In *Euro-Par'99 Parallel Processing*, Patrick Amestoy, Philippe Berger, Michel Daydé, Daniel Ruiz, Iain Duff, Valérie Frayssé, and Luc Giraud (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 422–429.
- [43] E. Gutiérrez, O. Plata, and E. L. Zapata. 2000. A compiler method for the parallel execution of irregular reductions in scalable shared memory multiprocessors. In *Proceedings of the 14th International Conference on Supercomputing (Santa Fe, New Mexico, USA) (ICS '00)*. Association for Computing Machinery, New York, NY, USA, 78–87. <https://doi.org/10.1145/335231.335239>
- [44] Eduardo José Gómez-Hernández, Juan M. Cebrian, Stefanos Kaxiras, and Alberto Ros. 2022. Splash-4: A Modern Benchmark Suite with Lock-Free Constructs. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 51–64. <https://doi.org/10.1109/IISWC55918.2022.00015>
- [45] Hwansoo Han and Chau-Wen Tseng. 1998. Improving compiler and run-time support for adaptive irregular codes. In *Proceedings. 1998 International Conference on Parallel Architectures and Compilation Techniques (Cat. No.98EX192)*. 393–400. <https://doi.org/10.1109/PACT.1998.727289>
- [46] Hwansoo Han and Chau-Wen Tseng. 2001. A comparison of parallelization techniques for irregular reductions. In *Proceedings 15th International Parallel and Distributed Processing Symposium. IPDPS 2001*. 8 pp.–. <https://doi.org/10.1109/IPDPS.2001.924963>
- [47] Timothy L. Harris. 2001. A Pragmatic Implementation of Non-blocking Linked-lists. In *Distributed Computing*, Jennifer Welch (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 300–314.
- [48] Arm Holdings. [n. d.]. AMBA 5 CHI Architecture Specification. <https://developer.arm.com/architectures/system-architectures/amba/amba-5>. [Online; 19-June-2025].
- [49] Arm Holdings. 2022. Armv9 Architecture Reference Manual for A-profile architecture. <https://developer.arm.com/documentation/ddi0602/2025-03/>. [Online; accessed 19-June-2025].
- [50] Arm Holdings. 2022. Do near or far atomics give the best performance on Neoverse systems? <https://developer.arm.com/documentation/ka004706/latest/>. [Online; accessed 19-June-2025].
- [51] Arm Holdings. 2025. The Armv9.6 architecture extension. https://developer.arm.com/documentation/109697/2025_03/Feature-descriptions/The-Armv9-6-architecture-extension. [Online; accessed 19-June-2025].
- [52] Libo Huang, Zhiying Wang, Nong Xiao, Yongwen Wang, and Qiang Dou. 2014. Integrated Coherence Prediction: Towards Efficient Cache Coherence on NoC-Based Multicore Architectures. *ACM Trans. Des. Autom. Electron. Syst.* 19, 3, Article 24 (June 2014), 22 pages. <https://doi.org/10.1145/2611756>
- [53] Intel. 2021. Intel® 64 and IA-32 Architectures Software Developer’s Manual. <https://cdrdv2.intel.com/v1/dl/getContent/671200>. [Online; accessed 19-June-2025].
- [54] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. 2010. High performance cache replacement using re-reference interval prediction (RRIP). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (Saint-Malo, France) (ISCA '10)*. Association for Computing Machinery, New York, NY, USA, 60–71. <https://doi.org/10.1145/1815961.1815971>
- [55] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. 2010. High performance cache replacement using re-reference interval prediction (RRIP). *SIGARCH Comput. Archit. News* 38, 3 (June 2010), 60–71. <https://doi.org/10.1145/1816038.1815971>
- [56] Ricardo Jesus and Michèle Weiland. 2023. AArch64 Atomics: Might They Be Harming Your Performance?. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (Montreal, QC, Canada) (PPoPP '23)*. Association for Computing Machinery, New York, NY, USA, 419–421. <https://doi.org/10.1145/3572848.3579838>
- [57] Ricardo Jesus and Michèle Weiland. 2023. A Study on the Performance Implications of AArch64 Atomics. In *High Performance Computing*, Abhinav Bhatel, Jeff Hammond, Marc Baboulin, and Carola Kruse (Eds.). Springer Nature Switzerland, Cham, 279–296.
- [58] Wookeun Jung, Jongsoo Park, and Jaejin Lee. 2014. Versatile and scalable parallel histogram construction. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*. 127–138. <https://doi.org/10.1145/2611756>

- [59] Alain Kägi, Doug Burger, and James R. Goodman. 1997. Efficient synchronization: let them eat QOLB. In *Proceedings of the 24th Annual International Symposium on Computer Architecture* (Denver, Colorado, USA) (ISCA '97). Association for Computing Machinery, New York, NY, USA, 170–180. <https://doi.org/10.1145/264107.264166>
- [60] Nodari Kankava. 2020. *Exploring the Efficiency of Multi-Word Compare-and-Swap*. Master's thesis. Uppsala University, Department of Information Technology.
- [61] R.E. Kessler and J.L. Schwarzmeier. 1993. Cray T3D: a new dimension for Cray Research. In *Digest of Papers. Compcon Spring*. 176–182.
- [62] David Klafneggger, Konstantinos Sagonas, and Kjell Winblad. 2014. Delegation Locking Libraries for Improved Performance of Multithreaded Programs. In *Euro-Par 2014 Parallel Processing*, Fernando Silva, Inês Dutra, and Vitor Santos Costa (Eds.). Springer International Publishing, Cham, 572–583.
- [63] David Koufaty and Josep Torrellas. 1998. Comparing data forwarding and prefetching for communication-induced misses in shared-memory MPs. In *Proceedings of the 12th International Conference on Supercomputing* (Melbourne, Australia) (ICS '98). Association for Computing Machinery, New York, NY, USA, 53–60. <https://doi.org/10.1145/277830.277847>
- [64] Sanjeev Kumar, Daehyun Kim, Mikhail Smelyanskiy, Yen-Kuang Chen, Jatin Chhugani, Christopher J. Hughes, Changkyu Kim, Victor W. Lee, and Anthony D. Nguyen. 2008. Atomic Vector Operations on Chip Multiprocessors. In *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA '08)*. IEEE Computer Society, USA, 441–452. <https://doi.org/10.1109/ISCA.2008.38>
- [65] George Kurian, Omer Khan, and Srinivas Devadas. 2013. The locality-aware adaptive cache coherence protocol. *SIGARCH Comput. Archit. News* 41, 3 (June 2013), 523–534. <https://doi.org/10.1145/2508148.2485967>
- [66] Andreas Kurth, Samuel Riedel, Florian Zaruba, Torsten Hoefler, and Luca Benini. 2020. ATUNs: Modular and Scalable Support for Atomic Operations in a Shared Memory Multiprocessor. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1109/DAC18072.2020.9218661>
- [67] An-Chow Lai and B. Falsafi. 1999. Memory sharing predictor: the key to a speculative coherent DSM. In *Proceedings of the 26th International Symposium on Computer Architecture (Cat. No.99CB36367)*. 172–183. <https://doi.org/10.1109/ISCA.1999.765949>
- [68] John H. Lau. 2021. *Chiplet Heterogeneous Integration*. Springer Singapore, Singapore, 413–439. https://doi.org/10.1007/978-981-16-1376-0_9
- [69] J. Laudon and D. Lenoski. 1997. The SGI Origin: A CcNUMA Highly Scalable Server. In *Proceedings of the 24th Annual International Symposium on Computer Architecture* (Denver, Colorado, USA) (ISCA '97). Association for Computing Machinery, New York, NY, USA, 241–251.
- [70] H. Q. Le, J. A. Van Norstrand, B. W. Thompto, J. E. Moreira, D. Q. Nguyen, D. Hrusecky, M. J. Genden, and M. Kroener. 2018. IBM POWER9 processor core. *IBM Journal of Research and Development* 62, 4/5 (2018), 2:1–2:12.
- [71] D. Lenoski, J. Laudon, K. Gharachorloo, W.-D. Weber, A. Gupta, J. Hennessy, M. Horowitz, and M.S. Lam. 1992. The Stanford Dash multiprocessor. *Computer* 25, 3 (1992), 63–79. <https://doi.org/10.1109/2.121510>
- [72] Jurij Leskovec, Deepayan Chakrabarti, Jon Kleinberg, and Christos Faloutsos. 2005. Realistic, Mathematically Tractable Graph Generation and Evolution, Using Kronecker Multiplication. In *Knowledge Discovery in Databases: PKDD 2005*, Alípio Mário Jorge, Luís Torgo, Pavel Brazdil, Rui Camacho, and João Gama (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 133–145.
- [73] Paul T. Lin, Michael A. Heroux, Richard F. Barrett, and Alan B. Williams. 2015. Assessing a mini-application as a performance proxy for a finite element method engineering application. *Concurrency and Computation: Practice and Experience* 27, 17 (2015), 5374–5389. <https://doi.org/10.1002/cpe.3587> <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.3587>
- [74] Gabriel H. Loh, Samuel Naffziger, and Kevin Lepak. 2021. Understanding Chiplets Today to Anticipate Future Integration Opportunities and Limits. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 142–145. <https://doi.org/10.23919/DAT51398.2021.9474021>
- [75] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 Simulator: Version 20.0+. [arXiv:2007.03152 \[cs.AR\]](https://arxiv.org/abs/2007.03152) <https://arxiv.org/abs/2007.03152>
- [76] Hao Luo, Jacob Brock, Pengcheng Li, Chen Ding, and Chencheng Ye. 2016. Compositional model of coherence and NUMA effects for optimizing thread and data placement. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 151–152. <https://doi.org/10.1109/ISPASS.2016.7482086>
- [77] Lorién López-Villellas, Rubén Langarita-Benítez, Asaf Badouh, Victor Soria-Pardos, Quim Aguado-Puig, Guillem López-Paradis, Max Doblas, Javier Setoain, Chulho Kim, Makoto Ono, Adrià Armejach, Santiago Marco-Sola, Jesús Alastruey-Benedé, Pablo Ibáñez, and Miquel Moreto. 2024. GenArchBench: A genomics benchmark suite for arm HPC processors. *Future Generation Computer Systems* 157 (2024), 313–329. <https://doi.org/10.1016/j.future.2024.03.050>
- [78] Pooria M. Yaghini, George Michelogiannakis, and Paul V. Gratz. 2019. SpecLock: Speculative Lock Forwarding. In *2019 IEEE 37th International Conference on Computer Design (ICCD)*. 273–282. <https://doi.org/10.1109/ICCD46524.2019.00041>
- [79] Jason Mars, Lingjia Tang, Robert Hundt, Kevin Skadron, and Mary Lou Soffa. 2011. Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations. In *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 248–259.
- [80] Edward M McCreight. 1985. The dragon computer system. *Microarchitecture of VLSI Computers* (1985), 83–101.
- [81] Paul E McKenney and John D Slingwine. 1998. Read-copy update: Using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems*, Vol. 509518. Citeseer, 509–518.
- [82] Maged M. Michael. 2004. Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation* (Washington DC, USA) (PLDI '04). Association for Computing Machinery, New York, NY, USA, 35–46. <https://doi.org/10.1145/996841.996848>
- [83] Daniel Molka, Daniel Hackenberg, Robert Schöne, and Matthias S. Muller. 2009. Memory Performance and Cache Coherency Effects on an Intel Nehalem Multiprocessor System. In *2009 18th International Conference on Parallel Architectures and Compilation Techniques*. 261–270. <https://doi.org/10.1109/PACT.2009.22>
- [84] Daniel Molka, Daniel Hackenberg, Robert Schöne, and Wolfgang E. Nagel. 2015. Cache Coherence Protocol and Memory Performance of the Intel Haswell-EP Architecture. In *2015 44th International Conference on Parallel Processing*. 739–748. <https://doi.org/10.1109/ICPP.2015.83>
- [85] Shubhendu S. Mukherjee and Mark D. Hill. 1998. Using prediction to accelerate coherence protocols. *SIGARCH Comput. Archit. News* 26, 3 (April 1998), 179–190. <https://doi.org/10.1145/279361.279386>
- [86] Malek Musleh and Vijay S. Pai. 2015. Automatic sharing classification and timely push for cache-coherent systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Austin, Texas) (SC '15). Association for Computing Machinery, New York, NY, USA, Article 13, 12 pages. <https://doi.org/10.1145/2807591.2807649>
- [87] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H. Loh, Mahesh Subramony, and Sean White. 2021. Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families: Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 57–70. <https://doi.org/10.1109/ISCA52012.2021.00014>
- [88] Neha Narula, Cody Cutler, Eddie Kohler, and Robert Morris. 2014. Phase Reconciliation for Contended In-Memory Transactions. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 511–524. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/narula>
- [89] Hamish Nicholson, Andreea Nica, Aunn Raza, Viktor Sanca, and Anastasia Ailamaki. 2024. Chaosity: Understanding Contemporary NUMA-Architectures. In *Performance Evaluation and Benchmarking*, Raghunath Nambiar and Meikel Poess (Eds.). Springer Nature Switzerland, Cham, 59–76.
- [90] Ruslan Nikolaev. 2019. A Scalable, Portable, and Memory-Efficient Lock-Free FIFO Queue. In *33rd International Symposium on Distributed Computing (DISC 2019)* (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 146), Jukka Suomela (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 28:1–28:16. <https://doi.org/10.4230/LIPIcs.DISC.2019.28>
- [91] Ruslan Nikolaev and Binoy Ravindran. 2022. wCQ: A Fast Wait-Free Queue with Bounded Memory Usage. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures* (Philadelphia, PA, USA) (SPAA '22). Association for Computing Machinery, New York, NY, USA, 307–319. <https://doi.org/10.1145/3490148.3538572>
- [92] Håkan Nilsson and Per Stenström. 1994. An adaptive update-based cache coherence protocol for reduction of miss rate and traffic. In *PARLE '94 Parallel Architectures and Languages Europe*, Costas Halatsis, Dimitrios Maritas, George Philokyprou, and Sergios Theodoridis (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 363–374.

- [93] Nvidia. 2024. NVIDIA Grace Hopper Superchip Architecture Whitepaper. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper?ncid=no-ncid> [Online; accessed 19-June-2025].
- [94] Nvidia. 2025. Parallel Thread Execution ISA. <https://docs.nvidia.com/cuda/parallel-thread-execution/>. [Online; accessed 19-June-2025].
- [95] D.K. Poulsen and Pen-Chung Yew Pen-Chung Yew. 1994. Data Prefetching and Data Forwarding in Shared Memory Multiprocessors. In *1994 International Conference on Parallel Processing Vol. 2*, Vol. 2. 280–280. <https://doi.org/10.1109/ICPP.1994.81>
- [96] R. Rajwar and J.R. Goodman. 2001. Speculative lock elision: enabling highly concurrent multithreaded execution. In *Proceedings. 34th ACM/IEEE International Symposium on Microarchitecture. MICRO-34*. 294–305. <https://doi.org/10.1109/MICRO.2001.991127>
- [97] U. Ramachandran, Gautam Shah, A. Sivasubramaniam, A. Singla, and I. Yanasak. 1995. Architectural Mechanisms for Explicit Communication in Shared Memory Multiprocessors. In *Supercomputing '95: Proceedings of the 1995 ACM/IEEE Conference on Supercomputing*. 62–62. <https://doi.org/10.1145/224170.224399>
- [98] E. Rosti, E. Smirni, T. D. Wagner, A. W. Apon, and L. W. Dowdy. 1993. The KSR1: experimentation and modeling of poststore. *SIGMETRICS Perform. Eval. Rev.* 21, 1 (June 1993), 74–85. <https://doi.org/10.1145/166962.166985>
- [99] S. L. Scott. 1996. Synchronization and Communication in the T3E Multiprocessor. In *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems* (Cambridge, Massachusetts, USA) (ASPLOS VII). Association for Computing Machinery, New York, NY, USA, 26–36.
- [100] Marco Siracusa, Victor Soria-Pardos, Francesco Sgherzi, Joshua Randall, Douglas J. Joseph, Miquel Moretó Planas, and Adrià Armejach. 2023. A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) (MICRO '23). Association for Computing Machinery, New York, NY, USA, 1332–1346. <https://doi.org/10.1145/3613424.3614284>
- [101] Andrew Sohn and Yuetsu Kodama. 1998. Load balanced parallel radix sort. In *Proceedings of the 12th International Conference on Supercomputing* (Melbourne, Australia) (ICS '98). Association for Computing Machinery, New York, NY, USA, 305–312. <https://doi.org/10.1145/277830.277903>
- [102] Victor Soria-Pardos, Adrià Armejach, Tiago Mück, Dario Suárez-Gracia, José Joao, Alejandro Rico, and Miquel Moretó. 2023. DynAMO: Improving Parallelism Through Dynamic Placement of Atomic Memory Operations. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 30, 13 pages. <https://doi.org/10.1145/3579371.3589065>
- [103] Victor Soria-Pardos, Adrià Armejach, Dario Suárez, Didier Martinot, Arnaud Grasset, and Miquel Moretó. 2025. FLAMA: Architecting Floating-Point Atomic Memory Operations for Heterogeneous HPC Systems. In *2025 28th Euromicro Conference on Digital System Design (DSD)*.
- [104] Isaac Sánchez Barrera. 2022. *Exploiting data locality in cache-coherent NUMA systems*. Ph.D. Dissertation. UPC, Departament d'Arquitectura de Computadors. <https://doi.org/10.5821/dissertation-2117-367546>
- [105] Charles P. Thacker and Lawrence C. Stewart. 1987. Firefly: a multiprocessor workstation. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems* (Palo Alto, California, USA) (ASPLOS II). Association for Computing Machinery, New York, NY, USA, 164–172. <https://doi.org/10.1145/36206.36199>
- [106] Shahar Timnat, Anastasia Braginsky, Alex Kogan, and Erez Petrank. 2012. Wait-free linked-lists. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (New Orleans, Louisiana, USA) (PPoPP '12). Association for Computing Machinery, New York, NY, USA, 309–310. <https://doi.org/10.1145/2145816.2145869>
- [107] J Hans Van Hateren and Arjen van der Schaaf. 1998. Independent component filters of natural images compared with simple cells in primary visual cortex. *Proceedings of the Royal Society of London. Series B: Biological Sciences* 265, 1394 (1998), 359–366.
- [108] Guru Venkataramani, Christopher J. Hughes, Sanjeev Kumar, and Milos Prvulovic. 2011. DeFT: Design space exploration for on-the-fly detection of coherence misses. *ACM Trans. Archit. Code Optim.* 8, 2, Article 8 (June 2011), 27 pages. <https://doi.org/10.1145/1970386.1970389>
- [109] Jiawei Wang, Diogo Behrens, Ming Fu, Lilith Oberhauser, Jonas Oberhauser, Jitang Lei, Geng Chen, Hermann Härtig, and Haibo Chen. 2022. BBQ: A Block-based Bounded Queue for Exchanging Data and Profiling. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 249–262. <https://www.usenix.org/conference/atc22/presentation/wang-jiawei>
- [110] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. 2019. Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks. *arXiv preprint arXiv:1909.01315* (2019).
- [111] Andrew Waterman and Krste Asanović. 2025. The RISC-V Instruction Set Manual Volume I: Unprivileged ISA. <https://github.com/riscv/riscv-isa-manual/releases/download/riscv-isa-release-f122839-2025-03-27/riscv-unprivileged.pdf>. [Online; accessed 19-June-2025].
- [112] Henry Wong, Anne Bracy, Ethan Schuchman, Tor M. Aamodt, Jamison D. Collins, Perry H. Wang, Gautham China, Ankur Khandelwal Groen, Hong Jiang, and Hong Wang. 2008. Pangaea: A tightly-coupled IA32 heterogeneous chip multiprocessor. In *2008 International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 52–61.
- [113] S.C. Woo, M. Ohara, E. Torrie, J.P. Singh, and A. Gupta. 1995. The SPLASH-2 programs: characterization and methodological considerations. In *Proceedings 22nd Annual International Symposium on Computer Architecture*. 24–36. <https://doi.org/10.1109/ISCA.1995.524546>
- [114] Carole-Jean Wu, Aamer Jaleel, Will Hasenplaugh, Margaret Martonosi, Simon C. Steely, and Joel Emer. 2011. SHiP: Signature-based Hit Predictor for high performance caching. In *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 430–441.
- [115] Chaoran Yang and John Mellor-Crummey. 2016. A wait-free queue as fast as fetch-and-add. *SIGPLAN Not.* 51, 8, Article 16 (Feb. 2016), 13 pages. <https://doi.org/10.1145/3016078.2851168>
- [116] Zhuoping Yang, Shixin Ji, Xingzhen Chen, Jinming Zhuang, Weifeng Zhang, Dharmesh Jani, and Peipei Zhou. 2024. Challenges and Opportunities to Enable Large-Scale Computing via Heterogeneous Chiplets. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 765–770. <https://doi.org/10.1109/ASP-DAC58780.2024.10473961>
- [117] Hao Yu and Lawrence Rauchwerger. 2000. Adaptive reduction parallelization techniques. In *ACM International Conference on Supercomputing 25th Anniversary Volume* (Munich, Germany). Association for Computing Machinery, New York, NY, USA, 311–322. <https://doi.org/10.1145/2591635.2667180>
- [118] Benlong Zhang, Junbin Kang, Tianyu Wo, Yuda Wang, and Renyu Yang. 2014. A Flexible and Scalable Affinity Lock for the Kernel. In *2014 IEEE Intl Conf on High Performance Computing and Communications, 2014 IEEE 6th Intl Symp on Cyberspace Safety and Security, 2014 IEEE 11th Intl Conf on Embedded Software and Syst (HPCC,CSS,ICSS)*. 34–37. <https://doi.org/10.1109/HPCC.2014.12>
- [119] Guowei Zhang, Webb Horn, and Daniel Sanchez. 2015. Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) (MICRO-48). Association for Computing Machinery, New York, NY, USA, 13–25. <https://doi.org/10.1145/2830772.2830774>
- [120] L. Zhang, Z. Fang, and J.B. Carter. 2004. Highly efficient synchronization based on active memory operations. In *18th International Parallel and Distributed Processing Symposium, 2004. Proceedings*. 58–. <https://doi.org/10.1109/IPDPS.2004.1302981>
- [121] Yicong Zhang, Mingyu Wang, Wangguang Wang, Yangzhan Mai, Haiqiu Huang, and Zhiyi Yu. 2024. Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs Through In-Cache Atomic Operations. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 671–685. <https://doi.org/10.1109/MICRO61859.2024.00056>